Technical Design Document — OrderEase

GitHub repository: `[ADD PUBLIC GITHUB URL BEFORE SUBMISSION]`

Live application: `[ADD DEPLOYED APP URL, OR WRITE NOT DEPLOYED]`

Client: Company A, Jabalpur

Project: Distributor Ordering Platform

1. Problem and scope

Retailer orders currently arrive through notebooks, WhatsApp text, handwritten photos and Hindi voice notes. The office re-enters these orders into Tally, while sales staff lack counter-level visibility into stock, retailer prices and schemes. OrderEase digitizes ordering and adds a human-reviewed capture path for unstructured messages.

The implementation covers all six requested capability groups: retailer ordering, salesman offline ordering, text/photo/voice capture, scheme calculation, order-desk operations including credit/Tally/van loading, and inactive-retailer reporting.

2. Architecture and choices

• **Frontend:** React + Vite, responsive CSS and a PWA manifest/service worker. React keeps the catalogue, cart and review flows interactive. Vite gives a small static build.

• **Backend:** Node.js + Express REST API. Business rules run server-side so client requests cannot choose prices, discounts or free quantities.

• **Database:** SQLite through sql.js for a self-contained reviewer demo with no database server installation. It persists to `server/data/orderease.sqlite`. For production or multi-instance deployment, migrate to PostgreSQL.

• **Authentication:** JWT sessions. Twilio Verify integration sends/checks real SMS OTP when credentials are configured; development mode uses a clearly labelled demo OTP. Production refuses demo OTP and requires a strong JWT secret.

• **AI/automation:** Optional OpenAI-compatible chat/vision and transcription APIs. Deterministic Hindi/Hinglish parser is the text fallback; browser Tesseract OCR and browser speech recognition provide local fallbacks. AI results are suggestions only and need human confirmation.

• **Offline:** Service worker caches the app shell; catalogue snapshots are cached locally; IndexedDB stores salesman orders. Sync sends an idempotency key and backend revalidates current stock, price and credit.

3. Data model

• `users`: retailer, salesman and order-desk identities.

• `salesman_assignments`: restricts a salesman to assigned retailers.

• `products`: SKU, name, unit, base price, stock cases and image.

• `retailer_prices`: per-retailer product price overrides.

• `retailer_credit`: credit limit and outstanding balance per retailer.

• `schemes`: product-linked free-goods or discount rule.

• `orders`: retailer, creator, status, totals, source, credit hold, loaded state and offline idempotency key.

• `order_items`: SKU/product, quantity, free quantity, unit price, line total and original AI phrase.

• `aliases`: retailer-specific corrected phrases mapped to product IDs.

Retailer catalogue and credit APIs derive the retailer identity from the authenticated token. Salesman APIs require a selected retailer to exist in `salesman_assignments`. Desk-only routes are protected by role middleware.

4. Main flows

Retailer ordering

Login → fetch retailer-specific catalogue/credit → select cases → backend fetches current price and scheme → calculates line totals → checks stock and credit → creates `pending_review` or `credit_hold` order → desk review → confirmation.

Repeat order

Retailer selects a previous order. Backend verifies ownership, reads previous SKU/quantity lines and creates a new order using current price, stock and schemes; previous totals are not blindly reused.

Salesman offline ordering

Salesman chooses an assigned retailer and loads cached catalogue. If network submission fails, order is stored in IndexedDB. When online, queued orders sync with unique local IDs. Backend validates current assignment, stock, price, scheme and credit. Duplicate sync requests return the already-created order instead of duplicating it.

AI order capture

Text is sent to the parser. Image can be interpreted by a configured vision model or OCR'd in browser. Audio upload can be transcribed by a configured speech API; browser speech recognition is an alternative. Parsed lines contain source phrase, SKU, quantity, unit, confidence and alternatives. Unclear units/products remain blocked. Retailer edits and confirms; desk can correct an SKU, and the original phrase becomes a retailer-specific alias for future parsing.

Scheme engine

`buy 10 get 1` computes free cases as `floor(quantity / threshold) × reward`. Slab discount applies the configured percentage when quantity meets the threshold. Backend calculates price and checks available stock including free goods. Tests cover threshold and invalid-quantity cases.

Order desk

Desk reviews orders, rejects or confirms. A credit hold requires credit account adjustment before confirmation. Confirmation rechecks stock including free goods, decrements inventory, and adds the order amount to outstanding credit. Desk exports confirmed lines to CSV, generates consolidated paid/free case loading lists, marks orders loaded and reviews retailers inactive for 14 days.

5. AI uncertainty and failure handling

• AI must use known catalogue SKUs; unknown or low-confidence lines require correction.

• Every extracted line displays its original phrase, confidence and alternatives where available.

• The unit is not guessed: order confirmation requires the reviewer to choose `case`.

• Empty/invalid model responses and provider errors are surfaced; the local text parser or OCR fallback is labelled.

• Photo OCR and speech recognition may misread handwriting/accent/noisy audio. The UI requires a person to inspect the text and each extracted line.

• No AI order is placed automatically.

6. UI and design decisions

The interface uses a restrained green/neutral palette, large product imagery, high-contrast prices, visible stock/scheme labels and large quantity controls. Retailers see the credit balance near the catalogue. AI review keeps original wording beside the selected SKU and confidence so corrections are quick. Desk workflows are table/list-oriented for scanning and exception handling. Layout adapts to narrow phone widths. Loading, error, empty and offline states are shown in the application.

7. Assumptions (explicit)

1. **GST:** The sample does not specify tax rates or whether prices include GST. Demo prices are treated as GST-inclusive and the UI says so; tax breakup is not calculated.

2. **Units:** Product order units are cases. `peti` in the supplied sample is interpreted as case; other ambiguous units require confirmation.

3. **OTP:** Demo OTP is acceptable for local reviewer setup; production uses Twilio Verify when configured.

4. **AI providers:** API credentials are not supplied. Text fallback is deterministic; vision/audio provider features activate only with a compatible API key. Browser OCR and microphone speech recognition are fallback options.

5. **Stock timing:** Stock is checked when an order is created and rechecked at desk confirmation. Offline orders do not reserve stock.

6. **Credit:** A confirmed order adds its total to outstanding credit. Credit limit/outstanding can be adjusted by desk users; payment collection itself is outside scope.

7. **Tally:** Export is a CSV handoff, not a direct Tally API integration.

8. **Van loading:** A loading list consolidates confirmed, not-yet-loaded orders; marking loaded is an operational status, not delivery tracking.

9. **Database:** SQLite is used for portability in a single-instance demo; PostgreSQL is recommended for production scale and concurrent multi-instance deployments.

8. Cut and next steps

Out of scope per brief: delivery tracking, in-app payments and GST e-invoicing. Before production: managed PostgreSQL, schema migrations, real OTP/provider secrets, API rate limiting, audit logs, backups, monitoring, stronger test coverage, image/audio storage policies, a proper Hindi OCR evaluation set, and direct Tally integration if client access becomes available.